

Circuit Design Competition

Table 3 (Main) – University of Central Florida

By: Matthew Giannacco, Matias Guillen, Ryan Liu, Rachel Ostrow, Eren Siegman



Table of Contents

Introduction	p. 3
Design Development Process	p. 4
Milestone #1: Motor Implementation for Water Pump Design	p. 5
Milestone #2: Safety Feature and Failure Detection Implementation	p. 9
Milestone #3: Water and Electrical Current Sensing System	p. 13
Milestone #4: Light Sensing System	p. 16
Schematic Design	p. 17
AI Prompts and Use Cases	P. 18
Reflection	p. 19

Introduction:

We were tasked with building an embedded control system that can control and monitor the water pump motor for the Aquila Aerospace's rocket launch pad, named the *Firebird-1*. We tackled 4 major milestones, the specifics of which are outlined in their respective sections.



Figure 1.0: AI Generated Image of Launch Pad Detection System

Team Structure and Responsibilities:

Member Name	Responsibility
Eren Siegman	Embedded Programmer
Ryan Liu	Circuit Designer and Embedded Helper
Matthew Giannacco	Circuit Designer and AI Consulter
Matias Guillen	Documenter
Rachel Ostrow	Documenter

Figure 1.1: Team Structure and Responsibilities

Design Development Process:

The design and development subsection will describe our team's design and thought process given the following equipment items. Our design process with each of the given challenges would follow a five phase structure of component identification, circuit design and implementation, physical design construction, testing and finally documentation and recording.

Design Level Specification Overhead View:

Component Identification
Circuit Design and Implementation in KiCAD
Physical Design Construction
Testing
Documentation and Recording

Figure 2.0: System level overview of circuit design competition design process.

Milestone #1: Motor Implementation for Water Pump Design:

Milestone 1 required us to assemble an embedded controller to wait 3 seconds after the motor is powered on and display the motors RPM values. We chose to use the LCD display to display our RPM values. After the motor is powered on, a green LED is lit up to show that the system is operating correctly.

The purpose of this is to allow the engineers to verify the motor is powered on and can ramp up to the required speed simulating the water deluge.

L293D Datasheet Reference:

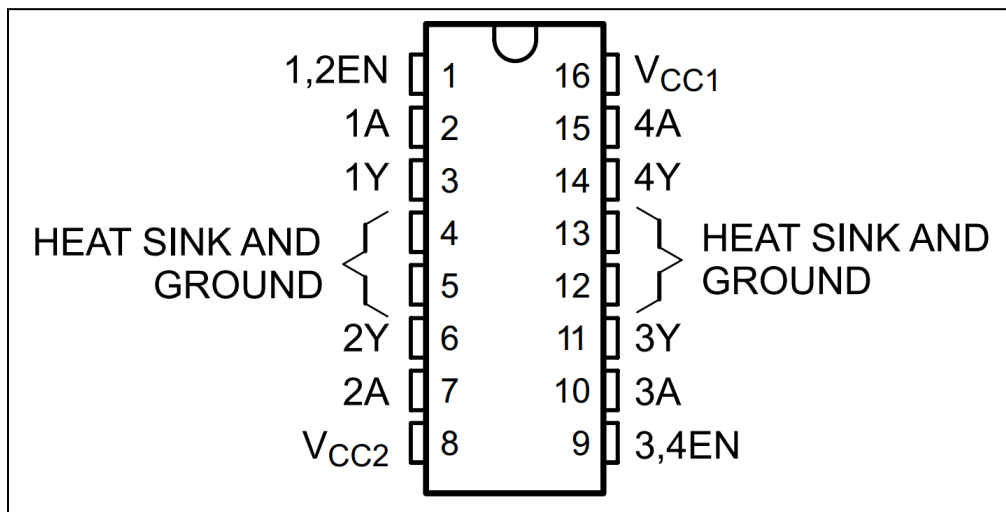


Figure 3.0: Display Datasheet Pinout for Motor Controller

L293D to DC Motor Pinout:

L293D Pin	Arduino Connection
1 (1,2EN)	D13
2 (1A)	D22
3 (1Y)	DC Motor Red Lead (+)
4 (GND)	GND
5 (GND)	GND
6 (2Y)	DC Motor Black Lead (-)
7 (2A)	D23

8 (VCC2)	MP+
9 (N/A)	NC
10 (3A)	NC
11 (3Y)	NC
12 (GND)	GND
13 (GND)	GND
14 (3Y)	NC
15 (4Y)	NC
16 (VCC1)	5V

*Figure 3.1: L293D Pinout Table***Encoder to Arduino Pinout:**

Encoder	Connection
1 (OUT)	D18
2 (VCC)	5V
3 (GND)	GND

*Figure 3.2: Encoder Pinout Table***Motor Driver State Diagram when EN = High (1):**

1A	2A	Output Behavior
Low (0)	Low (0)	Brake
Low (0)	High (1)	Clockwise Motion
High (1)	Low (0)	Counterclockwise Motion
High (1)	High (1)	Brake

Figure 3.3: Motor State Behavior

Circuit Design of Driver Circuit:

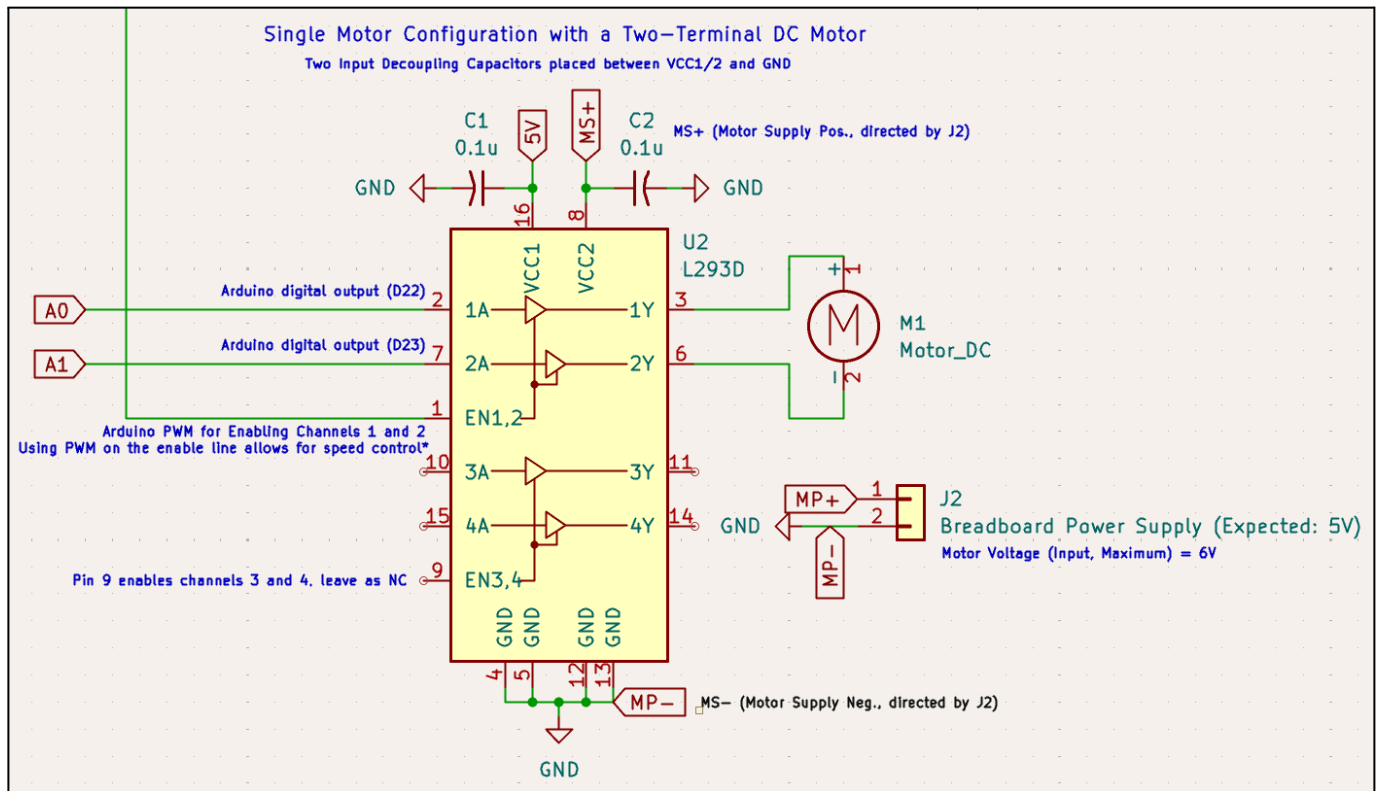


Figure 3.4: KiCAD Schematic Layout of Motor Controller with DC Motor

For a two-terminal DC motor connected to the L293D motor driver, we needed to use one channel for each terminal of the motor, totalling two. By leaving pin 9 open, channels 3 and 4 are disabled. First, we connected pin 1 to Arduino pin D13, a PWM GPIO pin. Using PWM on the enable line allows for motor speed control. Then, VCC for channel 1 needed to be connected to the Arduino 5V output, and VCC for channel 2 needed to be connected to the positive motor supply, named MP+. Next, pins 2 and 7 were both connected to an Arduino analog output. The state of these pins directs motor direction and impedance mode (figure nn). Lastly, Pin 3 is connected to the positive motor terminal, and pin 6 is connected to the negative motor terminal.

Circuit Design of the Motor Encoder:

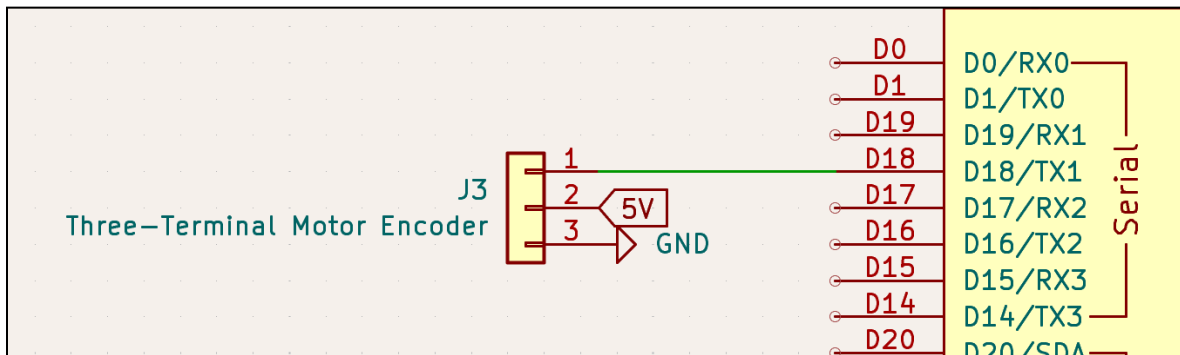


Figure 3.4: Motor Encoder KiCAD Schematic

Encoder setup only involved utilizing 3 pins labeled “J3” in the schematic. The encoder would ultimately provide our Arduino Mega 2560 with the correct information about its current status.

Physical Motor Controller Layout:

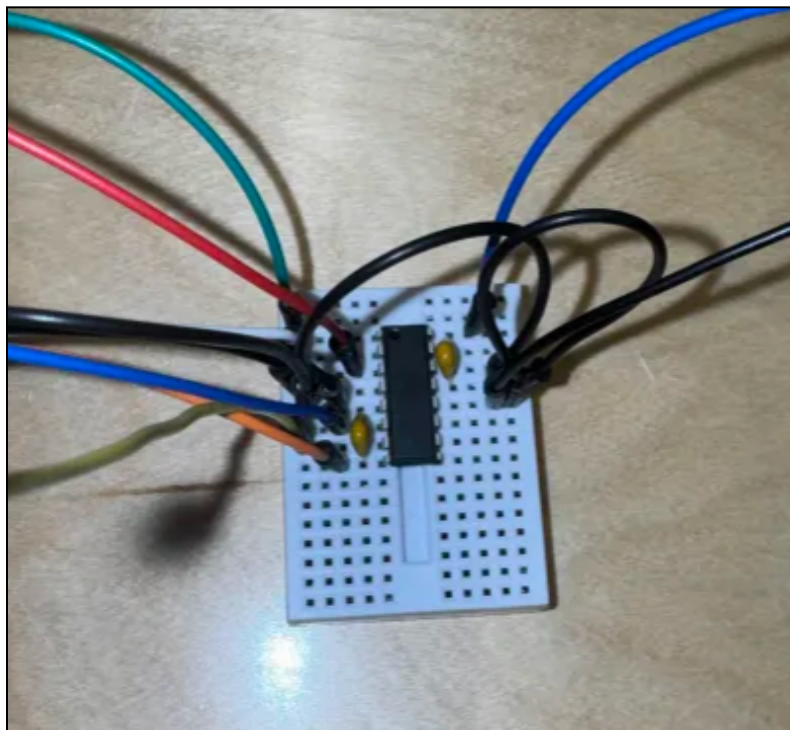


Figure 3.5: L293D Motor Driver Physical Breadboard

Image of motor driver shows the design implementation of the physical breadboard with L293D and two decoupling ceramic capacitors both at (0.1 μ F).

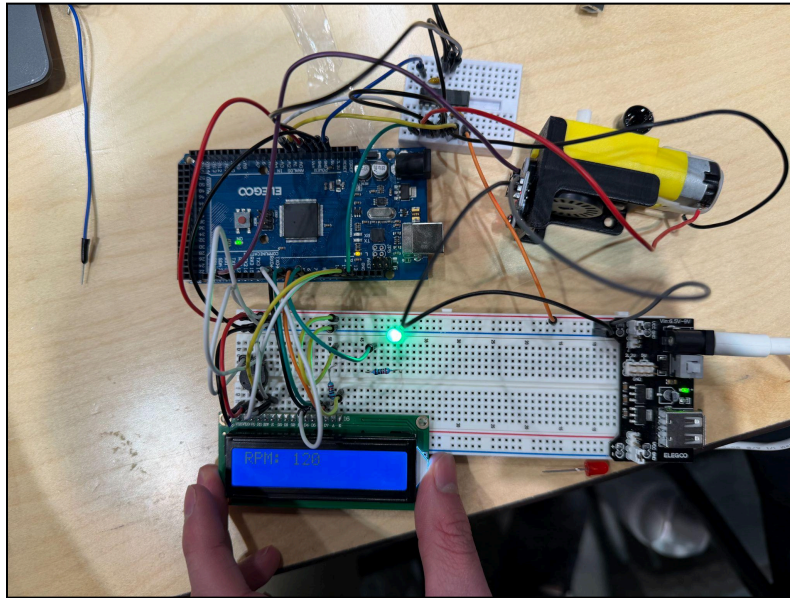
Operation Guide and Video:

Figure 3.6: Overall Design Image of Milestone 1 Implementation

<https://youtu.be/cxGc7zeUvSI>

For milestone 1 the implementation would provide the basis for the fundamental foundations of the water pump suppression system. The video above explained by Eren Siegman details various features and user operation. In summary, the system can be activated and includes these following features:

- When the power supply is activated, it will take 3 seconds to ramp up the motor speed. Once it has, the green LED is turned on.
- We used the L293D H-bridge IC motor driver to drive the motor.
- We connected a 2x16 LCD display to display the RPM through an interrupt and hardware timer.

Milestone #2: Safety Feature and Failure Detection Implementation:

Milestone 2 would further build on milestone 1 by implementing the following features.

- 3 second initial power on display with a countdown timer on LCD
- Motor ramp to final speed in 2 seconds after countdown
- Two LEDs implemented with green (turned on) and red (fault detection if full speed isn't reached in 75% after 2 seconds)
- After five seconds it will continue to attempt to reach, if not audible alarm will be set off (Optional Audible Alarm)

LCD Display Pins	Connection
1 (VSS)	GND
2 (VDD)	5V
3 (VO)	Potentiometer ADJ
4 (RS)	D12
5 (R/W)	GND
6 (E)	D11
7 (DB0)	NC
8 (DB1)	NC
9 (DB2)	NC
10 (DB3)	NC
11(DB4)	D2
12 (DB5)	D3
13 (DB6)	D4
14 (DB7)	D5
15 (A/VEE)	5V
16 (K)	GND
Pin A1	NC
Pin A2	NC
Pin A3	NC
Pin A4	NC

Figure 4.0: 16x2 LCD Pinout Table

Circuit Design of the 16x2 LCD Display:

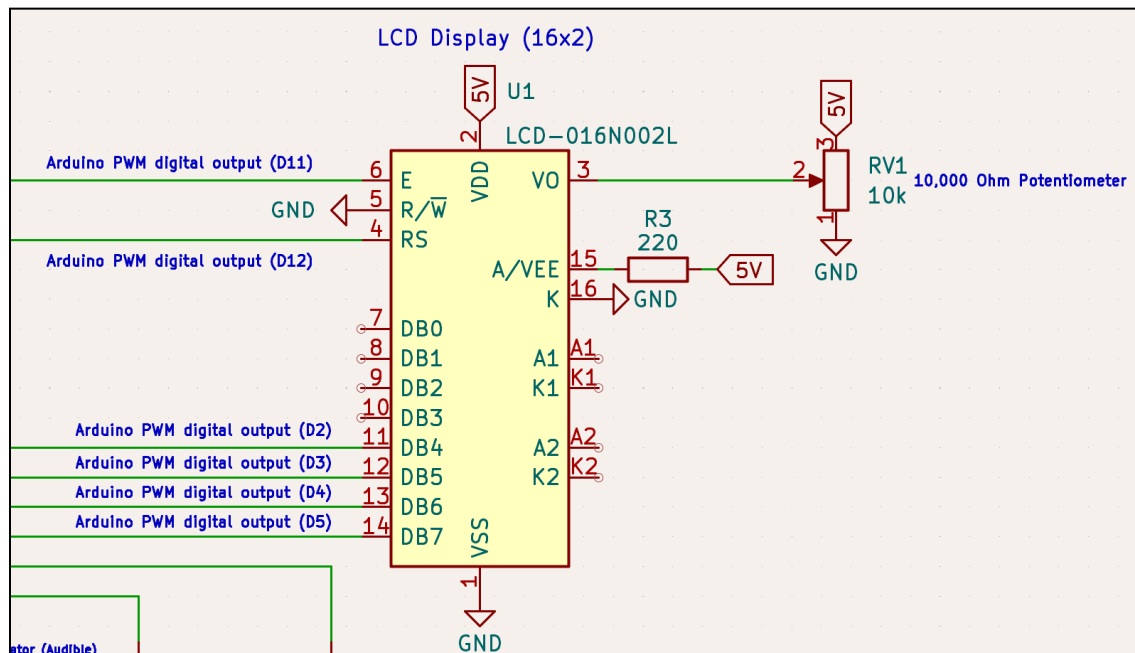


Figure 4.1: 16x2 LCD Display KiCAD Schematic

Circuit Design of the Failure Detection System:

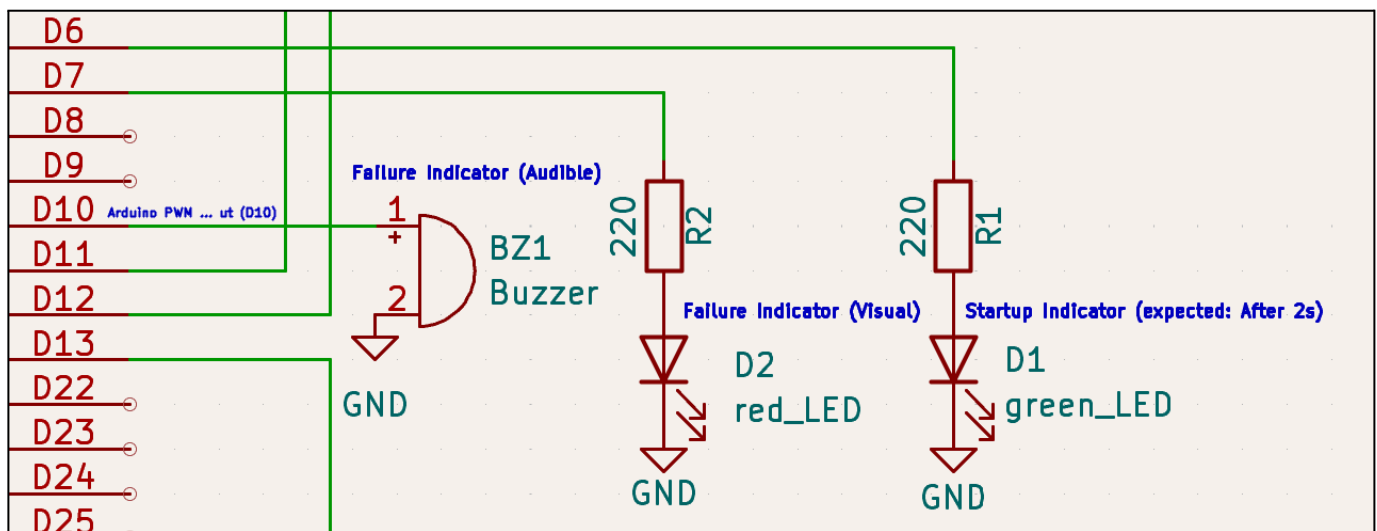


Figure 4.2: Failure detection system schematic, including a startup indicator, visual failure indicator, and audible failure indicator

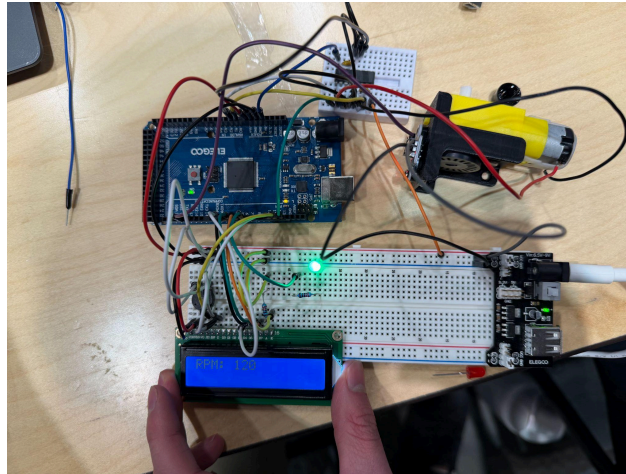
Operation Guide and Video:

Figure 4.3: Overall Design Image of Milestone 2 Implementation with Sound Sensor and Fault Detector

<https://www.youtube.com/watch?v=L5vJHLEFXgVAs>

Milestone 2 builds upon Milestone 1. The video shows the added features of the motor ramp up and fault detection. The fault detection works as follows:

- When everything works as normal, the motor ramps up speed over the course of 2 seconds.
- If the system notices after 2 seconds that the motor has not ramped up sufficiently, the fault is detected and activates the red LED and the buzzer simultaneously.
- If within 5 seconds the fault is cleared, the red LED and buzzer are switched off and the green LED turns on as normal.

Milestone #3: Water and Electrical Current Sensing System

Milestone 3 builds further upon milestone 2 in these ways:

- Add a water level sensor to check if water is available.
- If water is available, proceed normally
- If water isn't available, induce fault condition
- Add INA219 current sensor to report current on the LCD display

Circuit Design of the Water Sensor System:

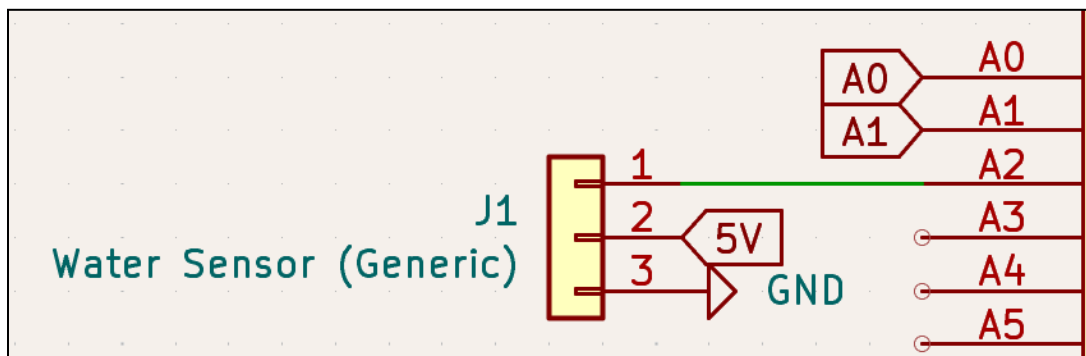
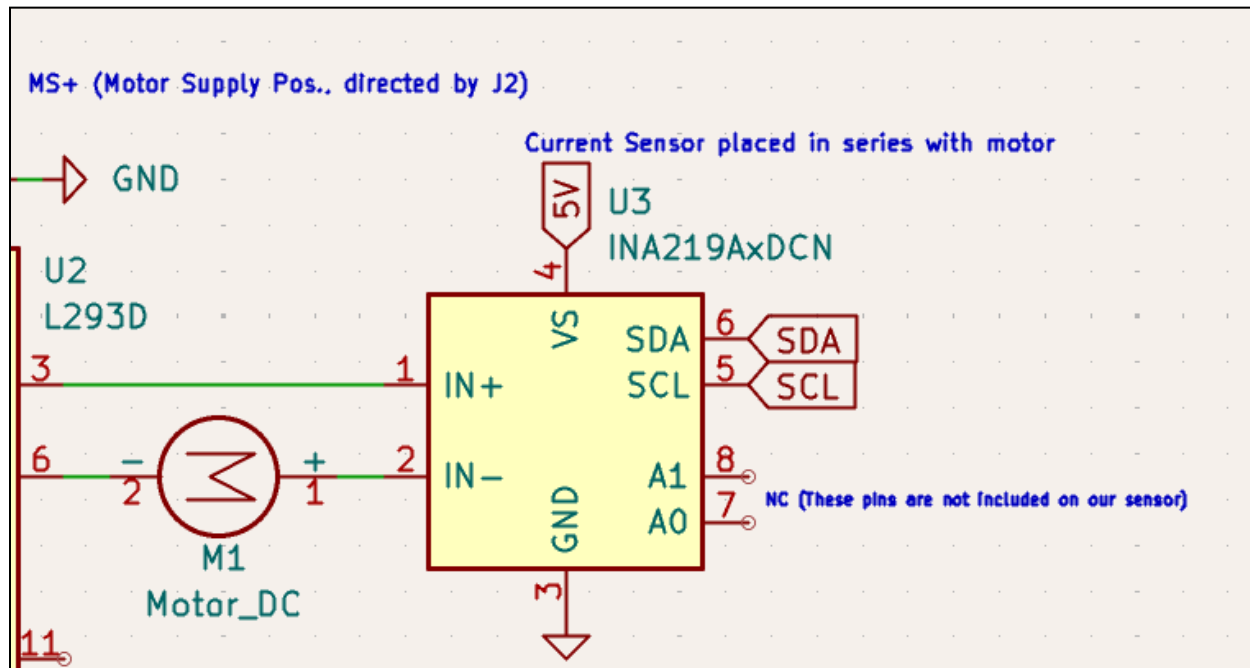


Figure 5.0: Water Sensor Schematic

Circuit Design of the Current Sensor:*Figure 5.1: Current Sensor Schematic*

Milestone 3 required us to add on a water level sensor and a INA219 current sensor to our pump system.

The water level sensor is to check if water is available. If it is, the operation proceeds. If it is not present, the fault condition is indicated with the red LED. If during normal operation water stops being detected, the fault condition is activated.

The motor's current is measured with the INA219 and reported on the LCD display. We conducted tests to determine the normal current level. If the current rises 50% above the normal level, a fault indication is indicated.

If fault condition is indicated, the motor is stopped, the red LED turned on, and the type of failure is displayed on the LCD display.

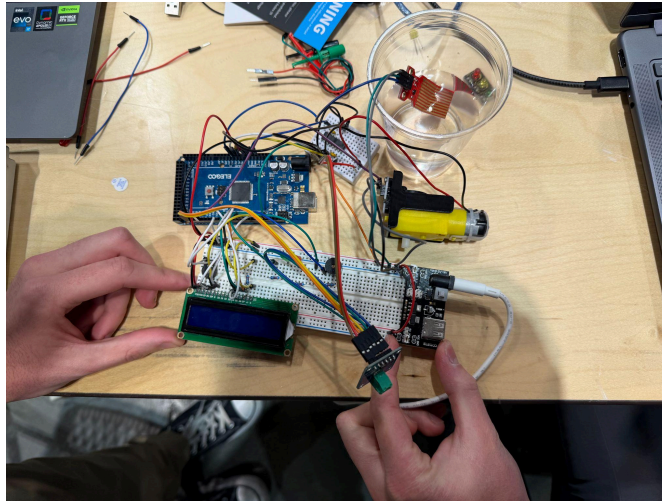
Operation Guide and Video:

Figure 5.2: Overall Design Image of Milestone 3 Implementation with water detection sensor and current sensor

https://www.youtube.com/watch/_02PJAcPJTU

This video shows our added features:

- Added the water and current sensors
- We realized we did not implement the countdown for milestone 2 so we implemented it
- Average current read is 200
- If current rises 50% above it, it will overcurrent and shut off
- If no water is detected when it first starts up, it will enter failure state

Milestone #4: Light Sensing System

Milestone 4 asked us to be creative and use additional sensors and parts to add extra fault detection and analysis. Operation Guide and Video:

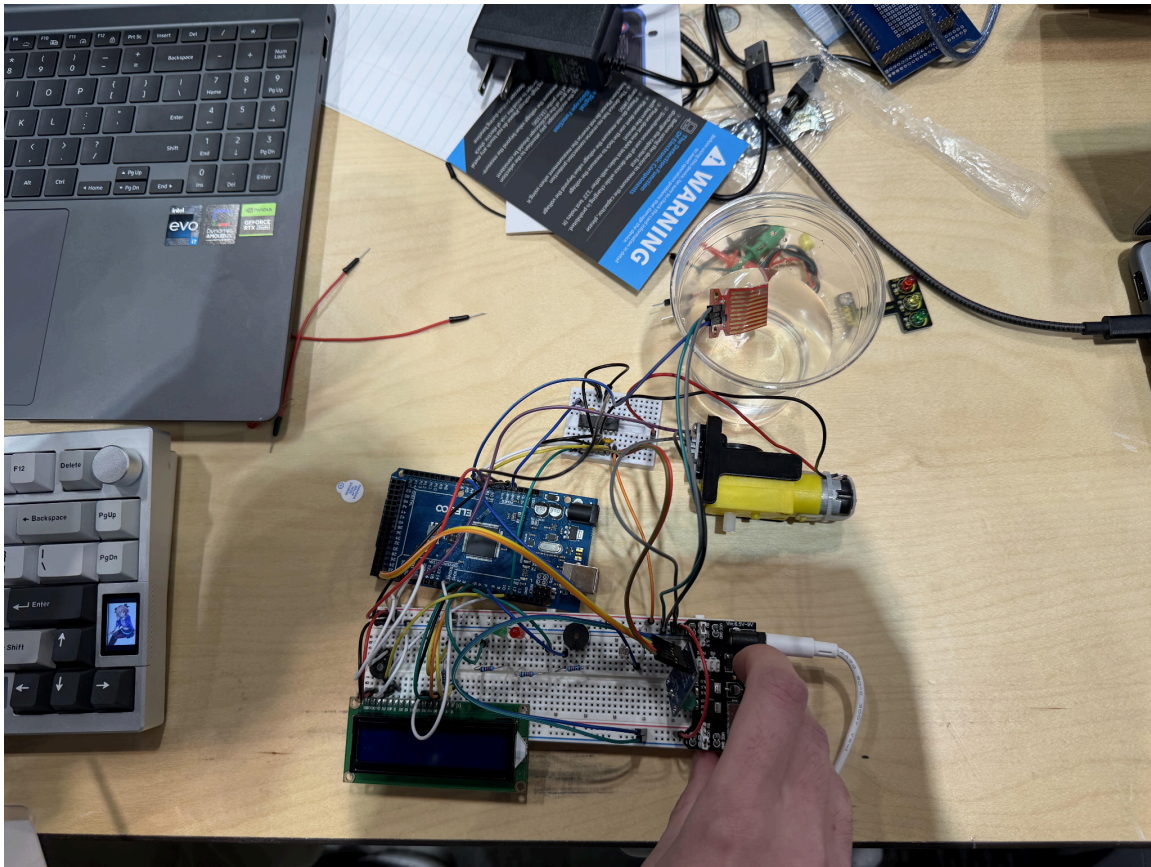


Figure 6.0: Overall Design Image of Milestone 4

<https://youtube.com/watch/xytbeGazgew>

This video explains:

We decided to add a light sensor. Because fire emits light, while the system is running, if the sensor doesn't detect bright enough light the motor won't turn on. If it does, the motor runs. This represents the water pump pumping water only when a fire is detected.

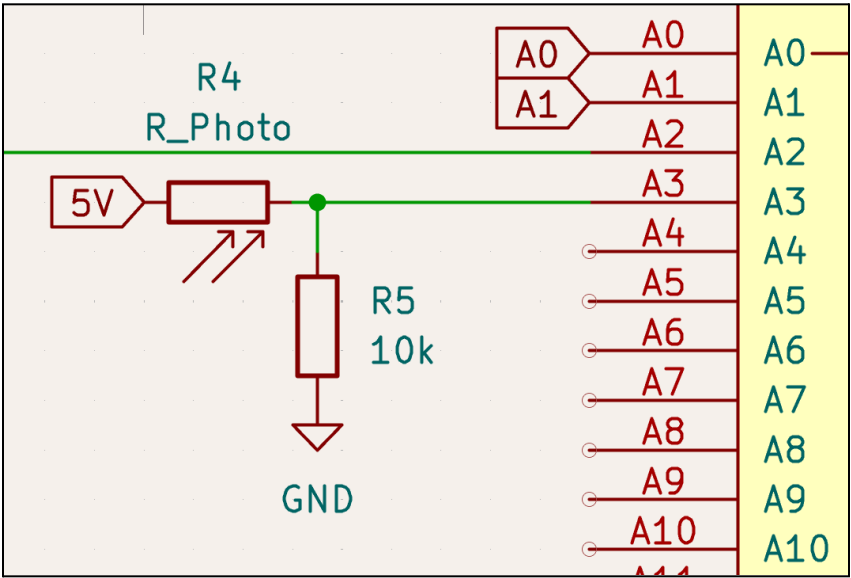


Figure 6.1: Overall Design Image of Milestone 4

Final Schematic Design:

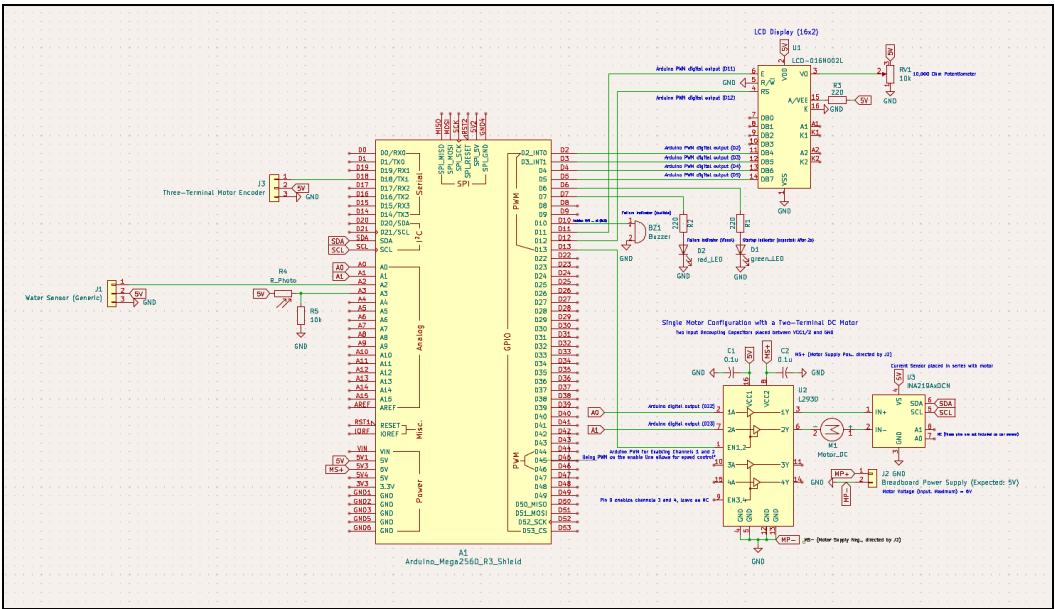


Figure 7.0: Final Schematic Design on KiCad

Final Physical Design:

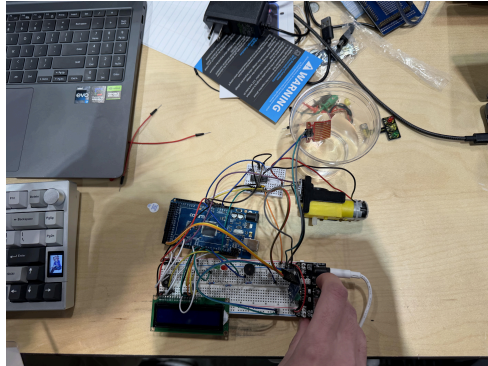


Figure 7.0: Final Physical Implementation

Final Arduino Code:

All files are included in a zip file uploaded in the folder.

AI Prompts and Use Cases:

<https://gemini.google.com/share/d53e23ae55fc>

Prompt log for image generation

<https://chatgpt.com/share/69b47e74-4cec-8003-b7eb-8643396ceb5b>

Prompt log for project advising, titled, “IEEE SEC Circuit Design Competition: Circuit Design and Embedded System Consultation”

For this contest, we created a custom GPT environment tailored for circuit design and embedded programming questions. In the instructions, we wrote, “Can advise and consult circuit design for Arduino. Includes but is not limited to datasheet analysis and code review. Answer any question, but advise the user if the request might break competition rules.”. Lastly, we supplied the agent with the three documents provided as homework assignments from IEEE, each datasheet, each Elegoo tutorial, each Elegoo code example and library, and the documentation/BOM of the starter kit.

The most important question we prompted our AI agent with regarding the motor driver IC. We supplied it with the datasheet for the L293D gate driver. Then, we asked for a datasheet analysis. We needed to know what I/O connections we could support the driver with from the Arduino. Lastly, we needed to know if we needed any additional electrical components, such as decoupling capacitors, diodes, or pulling resistors.

Reflection:

We are all glad to have had the opportunity to participate in this competition. This challenge has taught us a lot about the design process and working as a diversified team, as each person had a specialized area to focus on. Eren and Ryan worked as embedded programmers, Matthew worked on schematic design and prompt engineering, and Matias and Rachel focused on documenting and explaining the design and development process.

During this challenge, we discovered the most efficient way for our team to work was through a synchronous loop between updating schematics, embedded programming based on system design and communication and documentation.

Although everyone knew their designated role beforehand, tasks were flexible. Depending on what was most needed at the moment, tasks were distributed across the different members. Thinking about the real world applications presented helped us guide our decisions of how the system would be physically set up and implemented. We also learned how helpful AI is in this process.